1. Simple reflex agent for the Vacuum Cleaner world

```
class Main {
   public static void main(String[] args) {
      String A="Dirty";
      String B="Dirty";
      char location='A';

      for(int i=0;i<7;i++){
         String status=(location=='A')?A:B;

         if(status.equals("Dirty")){
            System.out.println(location + " -> Suck");

            if(location=='A'){
               A="Clean";
            }
            else{
               B="Clean";
            }
         }
         else if(location=='A'){
            System.out.println("A -> Right");
            location='B';
         }
         else{
            System.out.println("B -> Left");
            location='A';
         }
      }
   }
}
```

2. A* Search for 8 puzzle problem

```
import java.util.*;

class Main {

   static int h(int[] a) {
      int d = 0;
      for (int i = 0; i < 9; i++)
         if (a[i] != 0 && a[i] != i + 1) d++;
      return d;
   }

   static class Node {
      int[] a;
      int g;
      Node parent;

      Node(int[] a, int g, Node p) {
         this.a = a;
```

```
        this.g = g;
        parent = p;
    }
}

static void print(Node n) {
    if (n == null) return;
    print(n.parent);
    for (int i = 0; i < 9; i++) {
        System.out.print(n.a[i] + " ");
        if (i % 3 == 2) System.out.println();
    }
    System.out.println();
}

public static void main(String[] args) {

    int[] start = {1, 2, 7, 6, 0, 4, 3, 5, 8};
    int[] goal  = {1, 2, 3, 4, 5, 6, 7, 8, 0};

    PriorityQueue<Node> q = new PriorityQueue<>(
        (a, b) -> (a.g + h(a.a)) - (b.g + h(b.a))
    );

    Set<String> visited = new HashSet<>();

    q.add(new Node(start, 0, null));

    int[] moves = {-3, 3, -1, 1};

    while (!q.isEmpty()) {

        Node cur = q.poll();
        String key = Arrays.toString(cur.a);

        if (!visited.add(key)) continue;

        if (Arrays.equals(cur.a, goal)) {
            print(cur);
            System.out.println("Goal reached!");
            System.out.println("Total moves: " + cur.g);
            return;
        }

        int z = 0;
        while (cur.a[z] != 0) z++;

        for (int m : moves) {
            int n = z + m;

            if (n < 0 || n >= 9) continue;
```

```
            if (Math.abs(m) == 1 && z / 3 != n / 3) continue;

            int[] next = cur.a.clone();
            next[z] = next[n];
            next[n] = 0;

            if (!visited.contains(Arrays.toString(next)))
                q.add(new Node(next, cur.g + 1, cur));
        }
    }

    System.out.println("No solution!");
  }
}
```

3. Tower of Hanoi by State Space Search

```
import java.util.*;

public class Main {

  static void solve(int n) {

    Queue<String> q = new LinkedList<>();
    Map<String, String> parent = new HashMap<>();

    String start = "0".repeat(n), goal = "2".repeat(n);

    q.add(start);
    parent.put(start, null);

    while (!q.isEmpty()) {

      String s = q.poll();

      if (s.equals(goal)) {
        List<String> path = new ArrayList<>();

        while (s != null) {
          path.add(s);
          s = parent.get(s);
        }

        Collections.reverse(path);

        for (int i = 1; i < path.size(); i++) {
          String a = path.get(i - 1), b = path.get(i);

          for (int j = 0; j < n; j++)
```

```
                if (a.charAt(j) != b.charAt(j))
                    System.out.println("Move disk " + (j + 1) +
                        " from " + "ABC".charAt(a.charAt(j) - 48) +
                        " to " + "ABC".charAt(b.charAt(j) - 48));
            }

            System.out.println("Total moves: " + (path.size() - 1));
            return;
        }

        for (int i = 0; i < n; i++) {
            int from = s.charAt(i) - '0';

            boolean top = true;
            for (int j = 0; j < i; j++)
                if (s.charAt(j) - '0' == from) top = false;

            if (!top) continue;

            for (int to = 0; to < 3; to++) {
                if (to == from) continue;

                boolean valid = true;
                for (int j = 0; j < i; j++)
                    if (s.charAt(j) - '0' == to) valid = false;

                if (valid) {
                    char[] a = s.toCharArray();
                    a[i] = (char) ('0' + to);
                    String next = new String(a);

                    if (!parent.containsKey(next)) {
                        parent.put(next, s);
                        q.add(next);
                    }
                }
            }
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter disks: ");
    int n = sc.nextInt();

    if (n > 0) solve(n);
    else System.out.println("Invalid number of disks.");

    sc.close();
}
```

```
}
```

4. Local Beam Search for The Traveling Salesman Problem

```
import java.util.*;

class Main {

   static int[][] d = {
      {0, 10, 15, 20},
      {10, 0, 35, 25},
      {15, 35, 0, 30},
      {20, 25, 30, 0}
   };

   static int cost(int[] r) {
      int sum = 0;
      for (int i = 0; i < 4; i++)
         sum += d[r[i]][r[(i + 1) % 4]];
      return sum;
   }

   static void beamSearch() {
      int k = 2;
      List<int[]> beam = new ArrayList<>();
      beam.add(new int[]{0, 1, 2, 3});
      beam.add(new int[]{0, 2, 1, 3});

      for (int step = 0; step < 5; step++) {
         List<int[]> next = new ArrayList<>();

         for (int[] r : beam) {
            for (int i = 1; i < 4; i++) {
               for (int j = i + 1; j < 4; j++) {
                  int[] a = r.clone();
                  int t = a[i]; a[i] = a[j]; a[j] = t;
                  next.add(a);
               }
            }
         }

         next.sort(Comparator.comparingInt(Main::cost));
         beam = new ArrayList<>();

         for (int i = 0; i < Math.min(k, next.size()); i++)
            beam.add(next.get(i));
      }

      int[] best = beam.get(0);
```

```
        System.out.println("Best Route:");
        for (int x : best)
            System.out.print((char)('A' + x) + " -> ");
        System.out.println("A");
        System.out.println("Minimum Distance: " + cost(best));
    }

    public static void main(String[] args) {
        beamSearch();
    }
}
```

5. BFS to solve 8 puzzle problem

```
import java.util.*;

class Main {

    static String goal = "123456780";

    static class Node {
        String s;
        Node parent;

        Node(String s, Node p) {
            this.s = s;
            parent = p;
        }
    }

    static void bfs(String start) {

        Queue<Node> q = new LinkedList<>();
        Set<String> visited = new HashSet<>();

        q.add(new Node(start, null));
        visited.add(start);

        int[] moves = {-3, 3, -1, 1};

        while (!q.isEmpty()) {

            Node cur = q.poll();

            if (cur.s.equals(goal)) {
                print(cur);
                System.out.println("Goal Reached!");
                return;
            }
```

```
            int z = cur.s.indexOf('0');

            for (int m : moves) {
                int n = z + m;

                if (n < 0 || n >= 9) continue;
                if (Math.abs(m) == 1 && z / 3 != n / 3) continue;

                char[] a = cur.s.toCharArray();
                a[z] = a[n];
                a[n] = '0';

                String next = new String(a);

                if (visited.add(next))
                    q.add(new Node(next, cur));
            }
        }

        System.out.println("No solution!");
    }

    static void print(Node n) {
        if (n == null) return;

        print(n.parent);

        for (int i = 0; i < 9; i++) {
            System.out.print(n.s.charAt(i) + " ");
            if (i % 3 == 2) System.out.println();
        }
        System.out.println();
    }

    public static void main(String[] args) {
        bfs("123406758");
    }
}
```

6. DFS to solve Water Jug Problem

```
class Main {

    static int A = 4, B = 3;
    static boolean[][] visited = new boolean[5][4];

    static boolean dfs(int x, int y) {
```

```
        if (x == 2 || y == 2) {
            System.out.println("(" + x + "," + y + ")");
            return true;
        }

        if (visited[x][y]) return false;
        visited[x][y] = true;

        int[][] next = {
            {A, y}, {x, B}, {0, y}, {x, 0},
            {x - Math.min(x, B-y), y + Math.min(x, B-y)},
            {x + Math.min(y, A-x), y - Math.min(y, A-x)}
        };

        for (int[] s : next) {
            if (dfs(s[0], s[1])) {
                System.out.println("(" + x + "," + y + ")");
                return true;
            }
        }

        return false;
    }

    public static void main(String[] args) {
        System.out.println("DFS Solution:");
        if (!dfs(0, 0))
            System.out.println("No solution!");
    }
}
```

7. Backtracking to solve 8-Queens problem

```
class Main {
    static int N = 8;
    static int[] board = new int[N];

    static boolean isSafe(int row, int col) {
        for (int i = 0; i < row; i++) {
            if (board[i] == col ||
                Math.abs(board[i] - col) == Math.abs(i - row))
                return false;
        }
        return true;
    }

    static boolean solve(int row) {
        if (row == N)
```

```
            return true;

        for (int col = 0; col < N; col++) {
            if (isSafe(row, col)) {
                board[row] = col;

                if (solve(row + 1))
                    return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        if (solve(0)) {
            System.out.println("Solution:");
            for (int i = 0; i < N; i++) {
                for (int j = 0; j < N; j++) {
                    System.out.print(board[i] == j ? "Q " : ". ");
                }
                System.out.println();
            }
        } else {
            System.out.println("No solution!");
        }
    }
}
```

8. Simple chatbot

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {

            System.out.print("You: ");
            String s = sc.nextLine().toLowerCase();

            if (s.contains("hello") || s.contains("hi")) {
                System.out.println("Bot: Hello! How can I help you?");
            }
            else if (s.contains("how are you")) {
                System.out.println("Bot: I am fine! Thank you.");
            }
```

```
            else if (s.contains("your name") || s.contains("name")) {
                System.out.println("Bot: I am a simple chatbot.");
            }
            else if (s.contains("help")) {
                System.out.println("Bot: Sure! How can I help you?");
            }
            else if (s.contains("bye") || s.contains("exit")) {
                System.out.println("Bot: Goodbye! Have a nice day.");
                break;
            }
            else {
                System.out.println("Bot: Sorry, I don't understand.");
            }
        }

        sc.close();
    }
}
```